

# Introduction to Programming

## Problem Sheet 4: Solutions

Solutions to Problem Sheet 4 (not from the repo; written separately).

No imports should be required. You can find the laws that a typeclass is required to follow by searching the name of the typeclass on Hoogle.

### Problem 1: 2x2 integer matrices as a Num instance

A 2x2 matrix is represented here as `Mat a b c d`, meaning

```
| a b |  
| c d |
```

Make this type an instance of `Num`, using ordinary matrix arithmetic:

```
(+) as matrix addition  
(*) as matrix multiplication
```

Do NOT define `abs` or `signum` (they have no sensible meaning here)

```
data Matrix = Mat Integer Integer Integer Integer  
  deriving (Eq, Show)  
  
instance Num Matrix where  
  Mat a b c d + Mat e f g h = Mat (a+e) (b+f) (c+g) (d+h)  
  Mat a b c d * Mat e f g h = Mat (a*e + b*g) (a*f + b*h)  
                                (c*e + d*g) (c*f + d*h)  
  negate (Mat a b c d) = Mat (negate a) (negate b) (negate c) (negate d)
```

```
-- Integer n maps to (nI), where I is the identity matrix  
fromInteger n = Mat n 0 0 n
```

### Problem 2: Complex numbers as Num and Fractional

A complex number  $a + ib$  is stored as `'Complex a b'`.

```
data Complex = Complex Double Double  
  deriving (Eq, Show)
```

Num:

```
(+), (*)      standard complex arithmetic  
abs z        |z|
```

Do NOT define `signum` for the `Num` instance below

```
instance Num Complex where
  Complex a b + Complex c d = Complex (a+c) (b+d)
  Complex a b * Complex c d = Complex (a*c - b*d) (a*d + b*c)
  negate (Complex a b) = Complex (negate a) (negate b)
  abs (Complex a b)     = Complex (sqrt (a*a + b*b)) 0
  fromInteger n         = Complex (fromInteger n) 0
```

Dividing by zero may produce NaN/Infinity parts (as Double does); no special handling is required.

```
instance Fractional Complex where
  recip (Complex a b) = let r = a*a + b*b in Complex (a/r) (negate b / r)
  fromRational q      = Complex (fromRational q) 0
```

### Problem 3: Reduced row echelon form

Given a matrix as a list of rows, return its reduced row echelon form.

Assume every row has the same length. The empty matrix and matrices with empty rows should be returned unchanged. Use only Eq and Fractional, so it works for Rational (exact) as well as Double.

```
rref :: (Eq a, Fractional a) => [[a]] -> [[a]]
rref m = go 0 0 m
  where
    width = case m of
      (row:_) -> length row
      []      -> 0
```

```
-- go r c rows: rows 0..r-1 already have their pivots; look for a pivot
-- for row r in column c.
go r c rows
  | r >= length rows || c >= width = rows
  | otherwise =
    case [i | i <- [r .. length rows - 1], rows !! i !! c /= 0] of
      [] -> go r (c + 1) rows      -- no pivot in this column
      (i : _) ->
        let swapped = swapRows r i rows
            pivRow   = map (/ (swapped !! r !! c)) (swapped !! r)
            clear k row
              | k == r    = pivRow
              | otherwise = zipWith (\x y -> x - (row !! c) * y) row pivRow
        in go (r + 1) (c + 1) (zipWith clear [0 ..] swapped)

swapRows i j rows
  | i == j    = rows
  | otherwise = [ pick k row | (k, row) <- zip [0 ..] rows ]
  where
    pick k row
      | k == i    = rows !! j
      | k == j    = rows !! i
      | otherwise = row
```